

Deep Dive — The Transformer

Part 03 — 🗣️ Natural Language Processing · 03-nlp/notes/02-transformers.md

If you understand this document, you understand the architecture behind every model in Modules 04–11. Read it slowly. Then implement `../code/attention_from_scratch.py`.

1. The problem the transformer solved

RNNs process sequences one step at a time. Two consequences: 1. **No parallelism across time.** Training on long sequences is slow, and GPUs sit idle. 2. **Information bottleneck.** Position 1's influence on position 1000 must survive 999 sequential updates.

Attention (Bahdanau 2014) let a decoder look back at *all* encoder states. The 2017 insight was blunt: **drop the recurrence entirely and keep only attention.** Every position can now attend to every other position in one parallel operation.

The price: attention is $O(n^2)$ in sequence length. Every long-context technique in Module 04 is an attempt to pay less of that price.

2. Scaled dot-product attention

Each token produces three vectors via learned projections:

- **Query (Q)** — "what am I looking for?"
- **Key (K)** — "what do I offer?"
- **Value (V)** — "what do I actually contribute if selected?"

```
Attention(Q, K, V) = softmax( Q KT / √dk ) V
```

Step by step, for a sequence of `n` tokens and head dimension `dk`:

```
1. scores = Q @ K.T           (n × n)   how well does each query match each key?
2. scores = scores / sqrt(d_k)      scale
3. scores = scores + mask          -inf where attention is forbidden
4. weights = softmax(scores)   (n × n)   each row sums to 1
5. output  = weights @ V       (n × d_v) a weighted average of value vectors
```

Why divide by $\sqrt{d_k}$

$Q \cdot K$ is a sum of `dk` products. If the components are roughly unit-variance and independent, the dot product has variance `dk` — so it grows with dimension. Large logits push softmax into a near-one-hot regime where

its gradient vanishes. Dividing by $\sqrt{d_k}$ restores unit variance and keeps gradients alive. **This is the single most-asked transformer interview question. Know the variance argument, not just the formula.**

The database analogy

Attention is a **soft dictionary lookup**. A hard lookup returns the value whose key matches exactly. Attention returns a *weighted blend* of all values, weighted by how well each key matches the query — and because it's differentiable, the model learns what to look for.

3. Multi-head attention

One attention operation learns one kind of relationship. Run h of them in parallel on lower-dimensional projections and concatenate:

```
head_i = Attention(X W_i^Q, X W_i^K, X W_i^V)      for i = 1..h
MHA(X) = Concat(head_1, ..., head_h) W^O
```

With $d_{model} = 512$ and $h = 8$, each head works in 64 dimensions, so the total compute is roughly the same as one full-width head. Empirically, different heads specialise: some track syntax, some track coreference, some attend to the previous token, and some (the "attention sink" heads) dump probability mass on the first token when they have nothing useful to do.

Modern variants — and why they exist

Variant	K/V heads	Purpose
MHA (2017)	h	the original
MQA (Multi-Query)	1 shared	shrinks the KV cache dramatically; some quality loss
GQA (Grouped-Query)	g groups, $1 < g < h$	the modern default — nearly MHA quality, near-MQA cache
MLA (Multi-head Latent)	compressed latent KV	further cache compression (DeepSeek-style)

All three later variants exist for **one reason: inference memory**. The KV cache, not the weights, is what limits your batch size at long context. See §8.

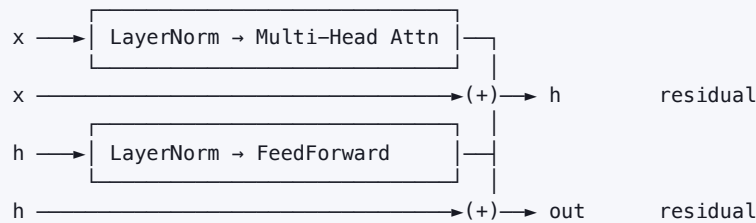
4. The masks

- **Causal (autoregressive) mask** — position i may only attend to positions $\leq i$. Implemented as an upper-triangular matrix of $-\infty$ added to the scores *before* softmax. This is what makes a decoder-only model able to train on all positions in parallel while never seeing the future.
- **Padding mask** — ignore padding tokens in a batch. Forget this and your model attends to garbage; the bug is silent and your metrics just get slightly worse.

Implementation note: add `-inf` (or a large negative number) *before* softmax. Zeroing weights *after* softmax is wrong — the remaining weights no longer sum to 1.

5. The transformer block

Modern (Pre-LN) form:



```
h = x + MHA(LayerNorm(x))
out = h + FFN(LayerNorm(h))
```

The feed-forward network

$$\text{FFN}(x) = W_2 \cdot \text{activation}(W_1 x + b_1) + b_2 \quad \text{with } d_{\text{ff}} = 4 \cdot d_{\text{model}}, \text{ classically}$$

Applied **independently to every position**. It holds roughly **two-thirds of the model's parameters** and is where most factual knowledge appears to be stored (the "key-value memory" interpretation). Modern LLMs use **SwiGLU**, which uses three matrices and typically sets $d_{\text{ff}} \approx (8/3) \cdot d_{\text{model}}$ to keep the parameter count comparable.

Pre-LN vs Post-LN

The original paper put LayerNorm *after* the residual add (Post-LN). It needs careful warmup and gets unstable with depth. Everything modern uses **Pre-LN** (normalise the sublayer input, keep the residual path clean end-to-end). Llama-class models additionally replace LayerNorm with **RMSNorm**.

6. Positional encoding — attention is order-blind

Self-attention is permutation-equivariant: shuffle the input and the outputs shuffle identically. Position information must be injected explicitly.

Scheme	How	Extrapolates?	Used by
Sinusoidal (2017)	fixed sin/cos of varying frequency, added to embeddings	weakly	original transformer
Learned absolute	a trainable vector per position	no — hard cap at trained length	BERT, GPT-2
ALiBi	a linear distance penalty added to attention scores	yes, gracefully	BLOOM, MPT
RoPE (Rotary)	<i>rotate</i> Q and K by an angle proportional to position	yes, with scaling tricks	Llama, Qwen, Mistral, most modern LLMs

RoPE, intuitively

Split each Q/K vector into 2-D pairs and rotate each pair by $m \cdot \theta_i$, where m is the position and θ_i is a per-pair frequency. Because a dot product between two rotated vectors depends only on the *difference* of their angles, attention becomes a function of **relative** position — for free, with no extra parameters.

Why you care: extending a model's context window is usually done by manipulating RoPE frequencies (position interpolation, NTK-aware scaling, YaRN). When a vendor says "we extended the context to 1M tokens", this is usually part of how. See Module 04 [04-11m].

7. The three architectural families

Family	Attention	Trained by	Best at	Examples
Encoder-only	bidirectional	masked language modelling	<i>understanding</i> : classification, NER, embeddings	BERT, RoBERTa, DeBERTa, ModernBERT
Decoder-only	causal	next-token prediction	<i>generation</i> , and in practice everything else	GPT, Llama, Claude, Qwen, Mistral
Encoder-decoder	bi- then causal + cross-attention	span corruption / seq2seq	transduction: translation, summarisation	T5, BART, FLAN-T5

Why decoder-only won: one simple objective, trivially parallel training, no architectural distinction between "input" and "output" (so in-context learning falls out for free), and it scales cleanly. Encoder-only models are *not* obsolete — they are still the right, cheap tool for embeddings and high-volume classification.

8. The KV cache — the thing that dominates inference economics

During generation, the model produces one token at a time. Naively you'd re-run attention over the whole prefix every step: $O(n^2)$ per token, $O(n^3)$ overall. Instead, cache the K and V vectors for every processed token and reuse them.

```
cache size = 2 × n_layers × n_kv_heads × d_head × seq_len × batch × bytes_per_param
```

Work a real example: 32 layers, 8 KV heads (GQA), head dim 128, 32k tokens, fp16, batch 1: $2 \times 32 \times 8 \times 128 \times 32,768 \times 2 \text{ bytes} \approx 4.3 \text{ GB}$ — for a single sequence. Multiply by your batch size.

Consequences you will meet in production: - **Prefill** (processing the prompt) is compute-bound and parallel; **decode** (generating) is memory-bandwidth-bound and sequential. They have completely different performance characteristics. - KV cache size, not parameter count, usually caps your concurrency. - This is why GQA/MQA/MLA exist, why **PagedAttention** (vLLM) was a breakthrough, and why **prompt caching** is offered by every major API.

9. Efficiency techniques worth knowing by name

- **FlashAttention** (1/2/3) — exact attention computed in a tiling pattern that never materialises the $n \times n$ matrix in HBM. 2–4× faster, dramatically less memory. Not an approximation — the math is identical.
- **Sliding-window / local attention** — each token attends to the last w tokens (Mistral, Longformer).
- **Sparse / block-sparse attention** — structured sparsity patterns.
- **Linear attention / SSMs** (Mamba, RWKV) — $O(n)$ sequence mixing; genuinely competitive, increasingly seen in **hybrid** stacks that interleave SSM and attention layers.
- **Mixture-of-Experts (MoE)** — replace the FFN with N experts and route each token to k of them. Parameter count grows without proportional compute per token. See Module 11 [11-llm-models].

10. Reading the shapes (the thing that actually trips people up)

For batch B , sequence n , model dim d , heads h , head dim $d_h = d/h$:

```
x          (B, n, d)
Q, K, V    (B, n, d)    → reshape/transpose → (B, h, n, d_h)
scores     (B, h, n, n)
weights     (B, h, n, n)
head output (B, h, n, d_h) → transpose/reshape → (B, n, d)
after W^0   (B, n, d)
```

The **(B, h, n, n)** tensor is the memory hog. At $B=8$, $h=32$, $n=8192$ in fp16 that's ~34 GB for one layer. That is precisely the tensor FlashAttention refuses to materialise.

11. Self-check before moving to Module 04

- [] Explain Q, K, V without using the word "attention".
- [] Justify $\sqrt{d_k}$ with the variance argument.
- [] Draw a Pre-LN block and say where each residual starts and ends.
- [] Explain why a causal mask lets you train on all positions at once.
- [] Compute a KV cache size for a given config, from memory.

- [] Say why GQA exists in one sentence.